

# BiblioIA: Sistema de Gestión de Biblioteca con Agente de Inteligencia Artificial"

---



- **Materia:** Base de Datos
- **Año / Carrera:** 3° año — Ing. en Sistemas de Información
- **Docentes:** Ing. Francisco Fazzio, Ing. Pablo Colombo
- **Ciclo lectivo:** 2026
- **Grupo :** Grupo Pandas
- **Integrantes :** Cheveste Baloni Ulises, Delfino Jeremias, Gallegos Marco, Ocampo Emmanuel

## 1. Introducción: descripción del problema y objetivos.

---

Este trabajo práctico integrador propone construir BiblioIA, un sistema de gestión de biblioteca que combina una base de datos relacional robusta con un agente de inteligencia artificial capaz de:

- Comprender preguntas formuladas en lenguaje natural (español).
- Traducirlas a consultas SQL válidas sobre el esquema de la biblioteca a través de un LLM.

- Ejecutar esas consultas y devolver resultados interpretables para el usuario.
- Generar recomendaciones de libros personalizadas basadas en el historial del usuario.

El proyecto integra los contenidos vistos a lo largo de la cursada: modelado relacional, DDL/DML avanzado, stored procedures y triggers, con el uso de APIs de LLMs (Large Language Models) accesibles libremente.

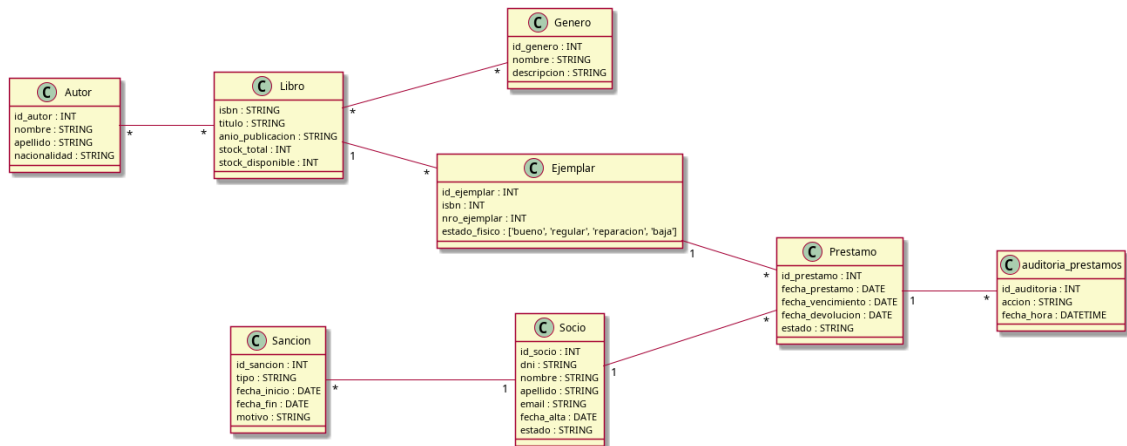
---

### ***Objetivos específicos:***

1. Diseñar e implementar el esquema relacional completo de una biblioteca usando DDL avanzado.
2. Incorporar restricciones de integridad (constraints, checks, claves foráneas) correctamente justificadas.
3. Implementar al menos 2 stored procedures y 2 triggers con lógica de negocio significativa.
4. Poblar la base de datos con datos realistas utilizando scripts DML.
5. Desarrollar un agente Text-to-SQL que convierta preguntas en lenguaje natural a consultas SQL.
6. Implementar un módulo de recomendaciones basado en patrones de préstamos.
7. Presentar una demo funcional en Jupyter Notebook o aplicación Python.

## 2. Base de Datos

### 2.1. Modelo Entidad-Relación (DER) completo con notación estándar.



## 2.2. Modelo Relacional (esquema de tablas con

## tipos y constraints).

---

```
create database if not exists biblioia;
use biblioia;
create table Genero (
    id_genero int auto_increment,
    nombre varchar(100) not null,
    descripcion text,
    constraint pk_genero primary key (id_genero),
    constraint uq_genero_nombre unique (nombre)
);
create table Autor (
    id_autor int auto_increment,
    nombre varchar(100) not null,
    apellido varchar(100) not null,
    nacionalidad varchar(100),
    constraint pk_autor primary key (id_autor)
);
create table Socio (
    id_socio int auto_increment,
    dni varchar(20) not null,
    nombre varchar(100) not null,
    apellido varchar(100) not null,
    email varchar(150) not null,
    fecha_alta date not null,
    estado varchar(20) not null default 'activo',
    constraint pk_socio primary key (id_socio),
    constraint uq_socio_dni unique (dni),
    constraint uq_socio_email unique (email),
    constraint chk_socio_estado check (estado in ('activo', 'suspendido', 'b
);
create table Libro (
    isbn varchar(20),
    titulo varchar(255) not null,
    anio_publicacion int,
    stock_total int not null default 0,
    stock_disponible int not null default 0,
    constraint pk_libro primary key (isbn),
    constraint chk_libro_stock check (stock_disponible >= 0 and stock_dispon
);

create table Libro_Genero (
    isbn varchar(20) not null,
    id_genero int not null,
    primary key (isbn, id_genero),
    constraint fk_libro_genero_libro
```

```

        foreign key (isbn)
        references Libro (isbn)
        on delete cascade,
    constraint fk_libro_genero_genero
        foreign key (id_genero)
        references Genero (id_genero)
        on delete cascade
);
create table Autor_libro (
    id_autor int,
    isbn varchar(20),
    constraint pk_autor_libro primary key (id_autor, isbn),
    constraint fk_autor_libro_autor foreign key (id_autor)
        references Autor(id_autor) on delete cascade,
    constraint fk_autor_libro_libro foreign key (isbn)
        references Libro(isbn) on delete cascade
);
create table Ejemplar (
    id_ejemplar int auto_increment,
    isbn varchar(20) not null,
    nro_ejemplar int not null,
    estado_fisico varchar(20) not null default 'bueno',
    constraint pk_ejemplar primary key (id_ejemplar),
    constraint uq_ejemplar_numero unique (isbn, nro_ejemplar),
    constraint chk_ejemplar_estado check (estado_fisico in ('bueno', 'regula
    constraint fk_ejemplar_libro foreign key (isbn)
        references Libro(isbn) on update cascade on delete cascade
);
create table Prestamo (
    id_prestamo int auto_increment,
    id_socio int not null,
    id_ejemplar int not null,
    fecha_prestamo date not null,
    fecha_vencimiento date not null,
    fecha_devolucion date null,
    estado varchar(20) not null default 'activo',
    constraint pk_prestamo primary key (id_prestamo),
    constraint chk_prestamo_estado check (estado in ('activo', 'devuelto', '
    constraint fk_prestamo_socio foreign key (id_socio)
        references Socio(id_socio) on update cascade on delete restrict,
    constraint fk_prestamo_ejemplar foreign key (id_ejemplar)
        references Ejemplar(id_ejemplar) on update cascade on delete restric
);
create table Sancion (
    id_sancion int auto_increment,
    id_socio int not null,
    tipo varchar(50) not null,

```

```

    fecha_inicio date not null,
    fecha_fin date not null,
    motivo text,
    constraint pk_sancion primary key (id_sancion),
    constraint fk_sancion_socio foreign key (id_socio)
        references Socio(id_socio) on update cascade on delete cascade
);

CREATE TABLE Auditoria_Prestamos (
    id_auditoria INT AUTO_INCREMENT,
    id_prestamo INT NOT NULL,
    accion VARCHAR(10) NOT NULL,
    fecha_hora DATETIME NOT NULL,
    usuario_bd VARCHAR(50) NOT NULL,
    CONSTRAINT chk_accion CHECK (accion IN ('insert', 'update', 'delete')),
    CONSTRAINT fk_prestamo FOREIGN KEY (id_prestamo) REFERENCES Prestamo(id_
    CONSTRAINT pk_auditoria PRIMARY KEY (id_auditoria)
);

create index idx_socio_dni on Socio(dni);
create index idx_socio_email on Socio(email);

```

## 2.3. Decisiones de diseño: justificación de tipos de datos, índices, políticas de FK.

---

La elección de los tipos de datos se realizó buscando la integridad de estos y una optimización del almacenamiento de los mismos.

Utilizamos INT AUTO\_INCREMENT en la mayoría de las entidades con el fin de agilizar los índices.

Además, implementamos restricciones CHECK y de unicidad bajo la cláusula CONSTRAINT (nombrando cada regla para facilitar la auditoría de errores), garantizando que la base de datos solo acepte valores lógicos y estructurados.

En cuanto a las políticas de claves foráneas, se aplicó ON DELETE CASCADE en las tablas intermedias y en la tabla sanción: si un libro, género o socio es eliminado, sus registros asociados en estas tablas se limpian automáticamente. Por otro lado, se utilizó ON DELETE RESTRICT en la tabla Préstamo; esto impide que se elimine a un socio o a un ejemplar del

sistema si todavía existen registros de préstamos activos asociados al mismo. Y por último, se configuró ON UPDATE CASCADE para asegurar que, si por alguna razón administrativa se modifica un identificador, el cambio se propague automáticamente a todas las tablas dependientes.

## 2.4. Descripción de cada stored procedure y trigger: propósito, parámetros, casos que maneja.

---

### 2.4.1. Procedimientos Almacenados (Stored Procedures)

#### SP\_REGISTRAR\_PRESTAMO

**Propósito:** Gestionar el alta de un nuevo préstamo en el sistema, asegurando que el socio y el ejemplar físico cumplan con las condiciones necesarias antes de concretar la operación.

**Parámetros:** IN par\_id\_socio INT (identificador único del socio), IN par\_id\_ejemplar INT (identificador único del ejemplar).

**Casos que maneja:** Valida que el socio exista y se encuentre activo. Se encarga de verificar que el socio no pueda tener más de tres préstamos activos en simultáneo, contabilizando los préstamos que se encuentran activos y vencidos a su nombre. Comprueba que el ejemplar que está por ser prestado, no se encuentre en reparación o haya sido dado de baja. Por último, valida que el ejemplar que está por prestarse se encuentre disponible en la biblioteca. Para esto el estado del préstamo que tiene asociado el ejemplar, debe ser 'devuelto'. Si alguna validación falla, se cancela la acción y retorna un error descriptivo. Caso contrario, el préstamo es registrado.

---

#### SP\_REGISTRAR\_DEVOLUCION

**Propósito:** Procesar el retorno de un ejemplar de la biblioteca, actualizando su disponibilidad y evaluando el cumplimiento de los plazos establecidos para aplicar penalizaciones si hicieran falta.

**Parámetros:** IN par\_id\_prestamo INT (identificador único del préstamo).

**Casos que maneja:** En primer instancia, verifica que el préstamo del cuál se quiere registrar su devolución aún no haya sido devuelto. Si el estado del préstamo ya se encuentra como 'vencido' o se ha excedido el plazo máximo de días, llama a 'sp\_generar\_sancion' para generar una sanción correspondiente al socio. Luego actualiza el estado del préstamo, calificándolo como 'devuelto' y cargando la fecha de devolución. Por último, actualiza el estado físico del ejemplar para que vuelva a estar disponible.

---

### **SP\_GENERAR\_SANCION**

**Propósito:** Automatizar el cálculo y registro de penalizaciones para los socios que incumplen las normas de la biblioteca.

**Parámetros:** IN id\_socio2 INT (identificador único del socio), IN tipo2 VARCHAR(50) (motivo de sanción), IN dias\_mora2 INT(días de retraso).

**Casos que maneja:** Calcula de manera dinámica la fecha de fin de la sanción proporcional a los días de mora. Carga el registro en la tabla "Sancion".

---

### **SP\_RENOVAR\_PRESTAMO**

**Propósito:** Renovar la fecha límite de devolución de un préstamo vigente.

**Parámetros:** IN id\_prestamo2 INT (identificador único del préstamo).

**Casos que maneja:** Valida que el préstamo se encuentre en estado 'activo'. No se pueden renovar préstamos vencidos o devueltos. Comprueba que el socio no posea sanciones activas en su cuenta. Actualiza la fecha de vencimiento sumando el plazo correspondiente establecido por la biblioteca.

---

**LibrosLeidos Propósito:** Devuelve los libros que el socio ya ha pedido

**Parámetros:** IN idSocio INT **Casos que maneja:** Usado exclusivamente por el módulo de Recomendaciones.

---

### **Recomendar**

**Propósito:** Mostrar libros que el socio no ha pedido prestados y podrían interesarle **Parámetros:** IN p\_id\_socio INT **Casos que maneja:** Es únicamente utilizado por la función de recomendar del proyecto.

---

## 2.4.2. Disparadores (Triggers)

### Triggers de Actualización de Stock

(trg\_actualizar\_stock\_insert, trg\_actualizar\_stock\_update)

**Propósito:** Mantener la coherencia automática entre el catálogo de libros y los movimientos físicos, garantizando que el campo 'stock\_disponible' de la tabla 'Libro' represente la realidad.

**Momento de ejecución:** AFTER INSERT, AFTER UPDATE sobre la tabla 'Prestamo'.

**Casos que maneja:** Cuando se realiza un nuevo préstamo, descuenta automáticamente una unidad del libro correspondiente. Cuando se realiza una devolución, aumenta automáticamente una unidad del libro correspondiente.

---

### Triggers de Actualización de Estado de Socio (trg\_estado\_socio)

**Propósito:** Automatizar el bloqueo de usuarios en el sistema al momento de recibir una penalización.

**Momento de ejecución:** AFTER INSERT sobre la tabla 'Sancion'.

**Casos que maneja:** Detecta la creación de una nueva sanción para un socio específico y actualiza el estado del socio, cambiando su estado en la tabla 'Socio'. Esto evita que el socio pueda sacar nuevos libros mediante el procedimiento de préstamos.

---

### Triggers de Auditoría

(trg\_audit\_prestamo\_insert, trg\_audit\_prestamo\_update)

**Propósito:** Registrar un historial de todos los movimientos y modificaciones realizados sobre la tabla 'Prestamo' por motivos de seguridad.

**Momento de ejecución:** AFTER INSERT, AFTER UPDATE sobre la tabla 'Prestamo'.

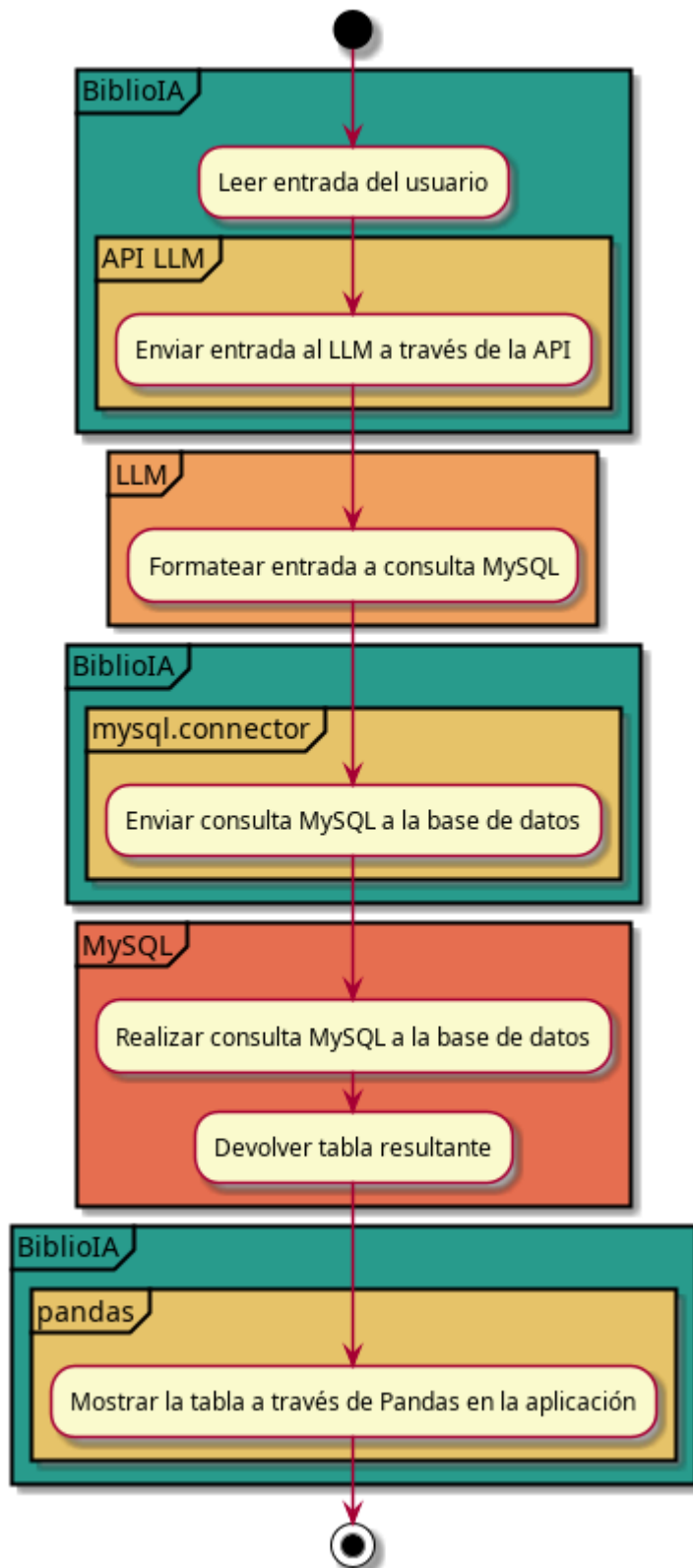
**Casos que maneja:** Cada vez que se presta, devuelve o actualiza un registro de préstamo, el trigger dispara una sentencia que guarda automáticamente la acción realizada en la tabla 'Auditoria\_Prestamos'. Se encarga de almacenar fecha y hora exactas, y el usuario de la base de datos que realizó la consulta.

## 3. Agente IA

---

### 3.1. Arquitectura: diagrama de flujo del

## proceso Text-to-SQL.



## **3.2. Prompt de sistema utilizado: versión final**

## con explicación de cada sección.

---

```
sys_prompt = """ Sos un Administrador de bases de datos de una biblioteca.
sabiendo que el schema de la base de datos en MySQL de esta biblioteca

1. **Genero** (Categorías de libros)
- id_genero (PK, auto_increment)
- nombre (VARCHAR 100, UNIQUE, NOT NULL)
- descripcion (TEXT)

2. **Autor** (Autores de libros)
- id_autor (PK, auto_increment)
- nombre (VARCHAR 100, NOT NULL)
- apellido (VARCHAR 100, NOT NULL)
- nacionalidad (VARCHAR 100)

3. **Socio** (Miembros de la biblioteca)
- id_socio (PK, auto_increment)
- dni (VARCHAR 20, UNIQUE, NOT NULL)
- nombre (VARCHAR 100, NOT NULL)
- apellido (VARCHAR 100, NOT NULL)
- email (VARCHAR 150, UNIQUE, NOT NULL)
- fecha_alta (DATE, NOT NULL)
- estado (VARCHAR 20, DEFAULT 'activo') - Valores: 'activo', 'suspendido'

4. **Libro** (Catálogo de libros)
- isbn (VARCHAR 20, PK)
- titulo (VARCHAR 255, NOT NULL)
- anio_publicacion (INT)
- stock_total (INT, DEFAULT 0)
- stock_disponible (INT, DEFAULT 0)
- id_genero (FK → Genero)

5. **Libro_genero** (Relación muchos-a-muchos entre Libros y Generos)
- id_genero (FK → Genero)
- isbn (FK → Libro)
- PK: (id_genero, isbn)

6. **Autor_libro** (Relación muchos-a-muchos entre Autores y Libros)
- id_autor (FK → Autor)
- isbn (FK → Libro)
- PK: (id_autor, isbn)

7. **Ejemplar** (Copias físicas de libros)
- id_ejemplar (PK, auto_increment)
- isbn (FK → Libro, NOT NULL)
- nro_ejemplar (INT, NOT NULL)
```

- estado\_fisico (VARCHAR 20, DEFAULT 'bueno') - Valores: 'bueno', 'regu

8. **\*\*Prestamo\*\*** (Registro de préstamos)

- id\_prestamo (PK, auto\_increment)
- id\_socio (FK → Socio, NOT NULL)
- id\_ejemplar (FK → Ejemplar, NOT NULL)
- fecha\_prestamo (DATE, NOT NULL)
- fecha\_vencimiento (DATE, NOT NULL)
- fecha\_devolucion (DATE, NULL)
- estado (VARCHAR 20, DEFAULT 'activo') - Valores: 'activo', 'devuelto'

9. **\*\*Sancion\*\*** (Sanciones a socios)

- id\_sancion (PK, auto\_increment)
- id\_socio (FK → Socio, NOT NULL)
- tipo (VARCHAR 50, NOT NULL)
- fecha\_inicio (DATE, NOT NULL)
- fecha\_fin (DATE, NOT NULL)
- motivo (TEXT)

INSTRUCCIONES IMPORTANTES:

- Cuando generes consultas SQL, asegúrate de que sean válidas para MySQL
- Usa INNER JOIN para relaciones requeridas y LEFT JOIN cuando sea necesario
- Siempre valida que los tipos de datos sean correctos.
- Tus respuestas deben contener únicamente la consulta SQL, sin ningún

EJEMPLOS DE CONSULTAS EN LENGUAJE NATURAL Y RESPUESTAS EN SQL:

Para la consulta:

Muéstrame todos los libros de Franz Kafka que se hayan publicado luego

Una respuesta en SQL adecuada sería:

```
SELECT l.*
FROM Libro l
INNER JOIN Autor_libro al ON l.isbn = al.isbn
INNER JOIN Autor a ON al.id_autor = a.id_autor
WHERE a.nombre = 'Franz'
AND a.apellido = 'Kafka'
AND l.anio_publicacion > 1950;
```

Para la consulta:

Muéstrame todos los autores que hayan publicado por lo menos 5 libros d

Una respuesta en SQL adecuada sería:

```
SELECT a.id_autor, a.nombre, a.apellido
FROM Autor a
INNER JOIN Autor_libro al ON a.id_autor = al.id_autor
INNER JOIN Libro_genero lg ON al.isbn = lg.isbn
INNER JOIN Genero g ON lg.id_genero = g.id_genero
```

```
WHERE g.nombre = 'Ciencia Ficción'
GROUP BY a.id_autor, a.nombre, a.apellido
HAVING COUNT(DISTINCT al.isbn) >= 5;

""
```

### 3.3. Evaluación del agente: 10+ preguntas, SQL generado, resultado y análisis de errores.

#### 3.3.1 Pregunta 1

**Consulta:** "¿Cuáles son los 5 libros más prestados este año?" **SQL**

**Generado:**

```
SELECT l.isbn, l.titulo, COUNT(p.id_prestamo) AS total_prestamos
FROM Prestamo p
INNER JOIN Ejemplar e ON p.id_ejemplar = e.id_ejemplar
INNER JOIN Libro l ON e.isbn = l.isbn
WHERE YEAR(p.fecha_prestamo) = YEAR(CURDATE())
GROUP BY l.isbn, l.titulo
ORDER BY total_prestamos DESC
LIMIT 5;
```

**Resultado:**

|   | isbn          | titulo                           | total_prestamos |
|---|---------------|----------------------------------|-----------------|
| 0 | 9788499890944 | 1984                             | 8               |
| 1 | 9788435020471 | Yo, Robot                        | 7               |
| 2 | 9788499083209 | Fundación                        | 6               |
| 3 | 9789871081446 | Facundo: Civilización y Barbarie | 6               |
| 4 | 9789875666496 | El Aleph                         | 5               |

**Análisis de Errores:** No se ha encontrado ninguno.

#### 3.3.2 Pregunta 2

**Consulta:** "¿Qué socios tienen préstamos vencidos en este momento?" **SQL**

**Generado:**

```
SELECT DISTINCT s.id_socio, s.dni, s.nombre, s.apellido, s.email
FROM Socio s
INNER JOIN Prestamo p ON s.id_socio = p.id_socio
WHERE p.estado = 'vencido' OR (p.fecha_vencimiento < CURDATE() AND p.fecha_
```

#### Resultado:

|    | id_socio | dni      | nombre    | apellido  | email                |
|----|----------|----------|-----------|-----------|----------------------|
| 0  | 26       | 32349012 | Victoria  | Dominguez | vdominguez@yahoo.com |
| 1  | 27       | 33450123 | Gonzalo   | Vega      | gvega@gmail.com      |
| 2  | 28       | 34561235 | Daniela   | Quiroga   | dquiroga@outlook.com |
| 3  | 29       | 35672346 | Ezequiel  | Luna      | eluna@gmail.com      |
| 4  | 30       | 36783457 | Antonella | Paz       | apaz@hotmail.com     |
| 5  | 1        | 41234567 | Marco     | Gallegos  | mgallegos@gmail.com  |
| 6  | 14       | 32345678 | Mariana   | Perez     | mperez@yahoo.com     |
| 7  | 15       | 38901234 | Agustín   | Sanchez   | asanchez@gmail.com   |
| 8  | 16       | 33456789 | Florencia | Romero    | fromero@outlook.com  |
| 9  | 17       | 39012345 | Joaquín   | Sosa      | jsosa@gmail.com      |
| 10 | 18       | 34561234 | Julietta  | Torres    | jtorres@hotmail.com  |
| 11 | 19       | 35672345 | Tomás     | Ruiz      | truiz@gmail.com      |
| 12 | 20       | 36783456 | Micaela   | Alvarez   | malvarez@yahoo.com   |
| 13 | 21       | 37894567 | Nicolás   | Gimenez   | ngimenez@gmail.com   |
| 14 | 22       | 38905678 | Carolina  | Castro    | ccastro@outlook.com  |
| 15 | 23       | 39016789 | Federico  | Molina    | fmolina@gmail.com    |

**Análisis de Errores** No se ha encontrado ninguno.

### 3.3.3 Pregunta 3

**Consulta:** "¿Cuántos ejemplares disponibles hay del libro con ISBN 978-03?" **SQL Generado:**

```
SELECT stock_disponible FROM Libro WHERE isbn = '978-03';
```

**Resultado:** Devuelve "stock\_disponible" vacío **Análisis de Errores:** No se ha encontrado ninguno.

### 3.3.4 Pregunta 4

**Consulta:** "¿Qué libros de ciencia ficción están disponibles para prestar?"

**SQL Generado:**

```
SELECT L.isbn, L.titulo, L.stock_disponible
FROM Libro L
INNER JOIN Genero G ON L.id_genero = G.id_genero
WHERE G.nombre = 'Ciencia Ficción' AND L.stock_disponible > 0;
```

**Resultado:**

|   | isbn          | titulo                | stock_disponible |
|---|---------------|-----------------------|------------------|
| 0 | 9788499890951 | Rebelión en la granja | 3                |

**Análisis de Errores:** No se ha encontrado ninguno.

### 3.3.5 Pregunta 5

**Consulta:** "¿Cuál es el historial de préstamos del socio con DNI 42345678?"

**SQL Generado:**

```
SELECT p.id_prestamo, l.titulo, p.fecha_prestamo, p.fecha_vencimiento, p.fecha_devolucion, p.estado
```

**Resultado:**

|   | id_prestamo | titulo          | fecha_prestamo | fecha_vencimiento | fecha_devolucion | estado   |
|---|-------------|-----------------|----------------|-------------------|------------------|----------|
| 0 | 2           | La Metamorfosis | 2025-09-01     | 2025-09-16        | 2025-09-15       | devuelto |
| 1 | 32          | Yo, Robot       | 2026-06-07     | 2026-06-22        | None             | activo   |

**Análisis de Errores:** No se ha encontrado ninguno.

### 3.3.6 Pregunta 6

**Consulta:** "¿Qué autores tienen más de 3 libros en la biblioteca?" **SQL**

**Generado:**

```
SELECT a.id_autor, a.nombre, a.apellido, COUNT(al.isbn) AS cantidad_libros
FROM Autor a
INNER JOIN Autor_libro al ON a.id_autor = al.id_autor
GROUP BY a.id_autor, a.nombre, a.apellido
HAVING COUNT(al.isbn) > 3;
```

**Resultado:** Devuelve una fila vacía.

**Análisis de Errores:** No se ha encontrado ninguno.

### 3.3.7 Pregunta 7

**Consulta:** "¿Cuántas sanciones se generaron en el último mes?" **SQL Generado:**

```
SELECT COUNT(*) AS total_sanciones FROM Sancion WHERE fecha_inicio >= DATE_
```

**Resultado:** Devuelve "total\_sanciones"=0

**Análisis de Errores:** No se ha encontrado ninguno.

### 3.3.8 Pregunta 8

**Consulta:** ¿Qué libros nunca han sido prestados desde que ingresaron al catálogo? **SQL Generado:**

```
SELECT l.isbn, l.titulo
FROM Libro l
WHERE NOT EXISTS (
    SELECT 1
    FROM Ejemplar e
    INNER JOIN Prestamo p ON e.id_ejemplar = p.id_ejemplar
    WHERE e.isbn = l.isbn
);
```

|   | isbn          | titulo                         |
|---|---------------|--------------------------------|
| 0 | 9786073206031 | Ingeniería de Software         |
| 1 | 9786073213114 | Redes de Computadoras          |
| 2 | 9786073214036 | Sistemas Operativos Modernos   |
| 3 | 9788420674107 | La Peste                       |
| 4 | 9788420674251 | El Proceso                     |
| 5 | 9788441413818 | Cuentos Completos              |
| 6 | 9788467045338 | Asesinato en el Orient Express |
| 7 | 9788467045406 | Y no quedó ninguno             |
| 8 | 9789500511874 | El payador perseguido          |
| 9 | 9789688802308 | El lenguaje de programación C  |

**Resultado:**

**Análisis de Errores:** No se ha encontrado ninguno.

### 3.3.9 Pregunta 9

**Consulta:** ¿Cuál es el socio que más préstamos realizó en los últimos 6 meses? **SQL Generado:**

```
SELECT s.id_socio, s.dni, s.nombre, s.apellido, COUNT(p.id_prestamo) AS total_prestamos
FROM Socio s
INNER JOIN Prestamo p ON s.id_socio = p.id_socio
WHERE p.fecha_prestamo >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
GROUP BY s.id_socio, s.dni, s.nombre, s.apellido
ORDER BY total_prestamos DESC
LIMIT 1;
```

**Resultado:**

|   | id_socio | dni      | nombre | apellido | total_prestamos |
|---|----------|----------|--------|----------|-----------------|
| 0 | 10       | 31234567 | Sofía  | Martinez | 2               |

**Análisis de Errores:** No se ha encontrado ninguno.

### 3.3.10 Pregunta 10

**Consulta:** ¿Qué socios nunca han recibido una sanción desde que se registraron? **SQL Generado:**

```

SELECT s.id_socio, s.nombre, s.apellido, s.email,
       s.fecha_alta, s.estado
FROM Socio s
WHERE s.id_socio NOT IN (
    SELECT DISTINCT id_socio FROM Sancion
)
ORDER BY s.fecha_alta;

```

## Resultado:

|    | id_socio | nombre          | apellido  | email                     | fecha_alta | estado     |
|----|----------|-----------------|-----------|---------------------------|------------|------------|
| 0  | 1        | Marco           | Gallegos  | mgallegos@gmail.com       | 2024-02-15 | activo     |
| 1  | 2        | Facundo         | Dun       | fdun@gmail.com            | 2024-03-10 | activo     |
| 2  | 3        | Ignacio Nicolás | Picart    | ipicart@hotmail.com       | 2024-03-12 | activo     |
| 3  | 4        | Matias          | Valdez    | mvaldez@utn.edu.ar        | 2024-04-01 | activo     |
| 4  | 5        | Valentino       | Gussalli  | vgussalli@gmail.com       | 2024-05-20 | suspendido |
| 5  | 6        | Antonella       | Gomez     | antonella.gomez@gmail.com | 2024-06-11 | activo     |
| 6  | 7        | Juan            | Perez     | jperez@yahoo.com          | 2024-07-05 | activo     |
| 7  | 8        | Lucía           | Fernandez | lfernandez@gmail.com      | 2024-08-15 | activo     |
| 8  | 9        | Martín          | Rodriguez | mrodriguez@outlook.com    | 2024-09-02 | suspendido |
| 9  | 10       | Sofia           | Martinez  | smartinez@gmail.com       | 2024-10-10 | activo     |
| 10 | 11       | Alejandro       | Lopez     | alopez@gmail.com          | 2024-11-20 | activo     |
| 11 | 12       | Camila          | Gonzalez  | cgonzalez@hotmail.com     | 2024-12-05 | activo     |
| 12 | 13       | Lucas           | Diaz      | ldiaz@gmail.com           | 2025-01-15 | activo     |
| 13 | 14       | Mariana         | Perez     | mperez@yahoo.com          | 2025-02-20 | activo     |
| 14 | 15       | Agustín         | Sanchez   | asanchez@gmail.com        | 2025-03-10 | suspendido |
| 15 | 16       | Florencia       | Romero    | fromero@outlook.com       | 2025-04-05 | activo     |
| 16 | 17       | Joaquín         | Sosa      | jsosa@gmail.com           | 2025-05-12 | activo     |
| 17 | 18       | Julieta         | Torres    | jtorres@hotmail.com       | 2025-06-20 | activo     |
| 18 | 19       | Tomás           | Ruiz      | truiz@gmail.com           | 2025-07-15 | activo     |
| 19 | 20       | Micaela         | Alvarez   | malvarez@yahoo.com        | 2025-08-10 | activo     |
| 20 | 21       | Nicolás         | Gimenez   | ngimenez@gmail.com        | 2025-09-05 | activo     |
| 21 | 22       | Carolina        | Castro    | ccastro@outlook.com       | 2025-10-20 | suspendido |
| 22 | 23       | Federico        | Molina    | fmolina@gmail.com         | 2025-11-15 | activo     |
| 23 | 24       | Paula           | Rios      | prios@hotmail.com         | 2025-12-10 | activo     |
| 24 | 25       | Diego           | Silva     | dsilva@gmail.com          | 2026-01-05 | activo     |
| 25 | 26       | Victoria        | Dominguez | vdominguez@yahoo.com      | 2026-02-20 | activo     |
| 26 | 27       | Gonzalo         | Vega      | gvega@gmail.com           | 2026-03-15 | activo     |
| 27 | 28       | Daniela         | Quiroga   | dquiroga@outlook.com      | 2026-04-10 | activo     |
| 28 | 29       | Ezequiel        | Luna      | eluna@gmail.com           | 2026-05-05 | activo     |
| 29 | 30       | Antonella       | Paz       | apaz@hotmail.com          | 2026-06-01 | suspendido |

**Análisis de Errores:** No se ha encontrado ninguno

## 4. Módulo de recomendaciones: descripción del algoritmo y ejemplos de salida.

Para el módulo de recomendaciones, en primer lugar se pide ingresar el **id del socio**, y luego, a través de los procedure **LibrosLeídos** y **Recomendar**, se busca en la base de datos aquellos: libros que el socio ha pedido, y libros **similares** a los ya leídos por el socio que este aún no haya pedido prestados, respectivamente. Luego, estas dos tablas resultantes se envían en formato **JSON** junto con un **prompt** al **LLM** para que este elabore un **listado** con los libros similares, y justificaciones de por qué los recomienda al socio. Finalmente, se imprime el resultado en la consola.

### Prompt:

```
prompt_recomendacion = """ En base a los libros leídos de la siguiente tabl
"""+leídos+"""
, haz un listado que incluya cada libro la siguiente tabla (similares) json
"""+similares+"""
, y justifica por qué lo recomendarías,
teniendo en cuenta que los libros de la tabla de similares comparten género
El formato de la lista debe ser del estilo:
- Título del libro: justificación de la recomendación
"""
```

### 4.1. Ejemplo de salida

id\_socio = 1 Libros leídos:

| id | titulo            | generos                      | autor             |
|----|-------------------|------------------------------|-------------------|
| 0  | Fundación         | Literatura Clásica           | Franz Kafka       |
| 1  | Muerte en el Nilo | Historia y Cultura Argentina | Jorge Luis Borges |

Libros similares:

| id | isbn   | titulo                         | stock_disponible |
|----|--------|--------------------------------|------------------|
| 0  | 978-03 | 2001: Una odisea espacial      | 2                |
| 1  | 978-09 | Asesinato en el Orient Express | 3                |
| 2  | 978-04 | Cita con Rama                  | 2                |
| 3  | 978-17 | El Nombre de la Rosa           | 3                |
| 4  | 978-18 | El Péndulo de Foucault         | 2                |
| 5  | 978-13 | El Resplandor                  | 3                |
| 6  | 978-12 | El Sabueso de los Baskerville  | 2                |
| 7  | 978-11 | Estudio en Escarlata           | 2                |
| 8  | 978-15 | La Llamada de Cthulhu          | 2                |
| 9  | 978-02 | Yo, Robot                      | 2                |

Recomendaciones: Para realizar las recomendaciones, es importante señalar que la tabla de libros leídos presenta una mezcla en sus metadatos (por ejemplo, asocia *Fundación* a Franz Kafka y "Literatura Clásica", y *Muerte en el Nilo* a Jorge Luis Borges e "Historia y Cultura Argentina").

Tomando en cuenta tanto los datos proporcionados en tu JSON como los **autores y géneros reales** de estas obras (ya que *Fundación* es de ciencia ficción por Isaac Asimov y *Muerte en el Nilo* es una novela policial de Agatha Christie), aquí tienes el listado de recomendaciones de la tabla de similares:

- **Yo, Robot:** Se recomienda porque comparte autor real (Isaac Asimov) y género (ciencia ficción) con *Fundación*. Además, al igual que los autores mencionados en tu tabla (como Kafka o Borges), es una obra fundamental de la literatura universal.
- **Asesinato en el Orient Express:** Se recomienda porque comparte autora real (Agatha Christie) y género (misterio/policial) con *Muerte en el Nilo*, ofreciendo una estructura de intriga y resolución de crímenes muy similar.
- **2001: Una odisea espacial:** Se recomienda porque comparte el género de ciencia ficción con *Fundación*, explorando temas de evolución humana, tecnología y el espacio exterior.
- **Cita con Rama:** Se recomienda porque, al igual que *Fundación*, es un pilar de la ciencia ficción dura que aborda el contacto con lo

desconocido y la exploración de estructuras masivas en el universo.

- **El Sabueso de los Baskerville:** Se recomienda porque comparte el género de misterio y resolución de crímenes con *Muerte en el Nilo*, y se alinea perfectamente con la categoría de "Literatura Clásica" que tu tabla le asigna a *Fundación*.
- **Estudio en Escarlata:** Se recomienda por la misma razón que el anterior; es una obra cumbre del misterio (género de *Muerte en el Nilo*) y un exponente indiscutible de la "Literatura Clásica".

## 5. Conclusiones: lecciones aprendidas, limitaciones del agente, mejoras posibles.

---

Durante el desarrollo de este proyecto, pudimos profundizar en nuestras habilidades con MySQL, y comprender mejor la importancia de los tipos de datos, restricciones y procedures a la hora de operar con la base de datos. Además, los requerimientos para el manejo de LLM's del trabajo nos incentivaron a aprender a utilizar nuevas librerías y a coordinar el funcionamiento de las distintas partes del programa para crear un notebook funcional. Con respecto a las limitaciones del agente, el uso de tokens es la mayor restricción que tenemos a la hora de operar, puesto que no contamos con suscripción alguna que aumente su cantidad. Como mejoras posibles, pensamos en implementar una función que permita al agente realizar operaciones más variadas e importantes en la base de datos, como lo sería poder ingresar nueva información y modificar la ya existente. Para conseguir esto, tendríamos que trabajar árdamente en todos los controles posibles para evitar cambios no deseados y potencialmente peligrosos.

## 6. Bibliografía y recursos utilizados.

---

- <https://dagster.io/blog/python-environment-variables>
- <https://dev.to/jakewitcher/using-env-files-for-environment-variables-in-python-applications-55a1>
- <https://dev.mysql.com/doc/connector-python/en/connector-python-example-connecting.html>

- <https://ai.google.dev/gemini-api/docs/quickstart>
- <https://googleapis.github.io/python-genai/>
- <https://ipywidgets.readthedocs.io/en/stable/index.html>